Team 1

КУРСОВИЙ ПРОЄКТ · ASP.NET CORE

New Balance Shop

Інтернет-магазин спортивного одягу та взуття: каталог, кошик, замовлення, кабінет покупця й адмін-функції.

TEAM LEAD
Панчук Вадим Владиславович
alfa40957@gmail.com

FULL-STACK РОЗРОБНИК
Ноцколіні Рафаїл Львович
Backend · Frontend · DevOps · QA
vicitmceo@gmail.com

Репозиторій

Живий сайт

Захист курсового проєкту · 2026
github.com/vicitmceo/NewBalanceShop

Навіщо цей проєкт

Проблема — Рішення — Цільова аудиторія

New Balance Shop

ПРОБЛЕМА

Складно швидко знайти й купити потрібну пару

- Каталог на великих сайтах перевантажений: десятки розділів, акцій, банерів.
- Щоб оформити замовлення, часто змушують створювати акаунт.
- Розмір, колір і наявність видно не одразу.

Приклад 1. Людина хоче кросівки 42 розміру в сірому кольорі й губить 10 хвилин на фільтри та реєстрацію.
Приклад 2. Покупець кладе товар у кошик, закриває вкладку, і кошик зникає.

→

РІШЕННЯ

Простий магазин: каталог → кошик → замовлення

- Каталог з картками: назва, розмір, колір, ціна, категорія.
- Кошик працює в сесії, замовлення можна оформити без реєстрації.
- Кабінет покупця: профіль та історія замовлень.
- Адмін-функції: керування товарами та користувачами.
- Три мови інтерфейсу: UA / EN / FR.

→

ЦІЛЬОВА АУДИТОРІЯ

Хто користується

- **Покупці 16–40 років**, які носять спортивне взуття й одяг щодня.
- **Любителі бігу й тренувань**, яким важливі розмір і наявність.
- **Адміністратор магазину**, який керує каталогом і користувачами.

Докладніше про персони: слайд «Персони».

SRS: вимоги до системи

13 функціональних вимог + нефункціональні · файл SRS.md у репозиторії

New Balance Shop

Функціональні вимоги (FR) і стан реалізації

ID	ВИМОГА	СТАН
FR-01, 02	Перегляд списку товарів і одного товару	**Готово**
FR-03	Фільтр і пошук: категорія, бренд, розмір, ціна, назва	**Готово**
FR-04, 05, 06	Створення, оновлення, видалення товару	**Готово**
FR-07, 08	Кошик і оформлення замовлення без реєстрації	**Готово**
FR-09	Статус замовлення (New … Cancelled)	**Готово**
FR-10, 11	Кабінет покупця: профіль, історія замовлень	**Готово**
FR-12, 13	Адмін: список, блокування, видалення користувачів	**Готово**

13 / 13

FR ПОВНІСТЮ

219

АВТОТЕСТІВ ПРОХОДЯТЬ

Нефункціональні вимоги (NFR)

- **Продуктивність:** читання через AsNoTracking, DTO замість сутностей.
- **Масштабованість:** Clean Architecture, окремі проєкти, stateless-сервіси з DI.
- **Безпека:** хеш паролів PBKDF2, ролі на адмін-ендпоінтах (401/403), customerId із сесії, білий список CORS, rate limiting.
- **Надійність:** міграції EF Core, 219 автотестів, покриття 91,1%.
- **Зручність:** інтерфейс UA/EN/FR, адаптивна верстка, кошик без обов'язкової реєстрації.

UI Kit

Дизайн-токени та компоненти фронтенду (React, App.css)

New Balance Shop

Кольори

Red	Black	Gray 600
#cf0a2c	#111111	#5c5c5c
Gray 400	Gray 200	Gray 100
#8c8c8c	#e2e2e2	#f4f4f4

Типографіка

Заголовок 800

Заголовок розділу 20/800

Основний текст 13–15 px. Шрифт: Helvetica Neue, Helvetica, Arial, system-ui.

Кнопки

В кошик | Hover / Купити | Вийти

Навігація й мова

Каталог | Мої замовлення | UA

Форми та статуси

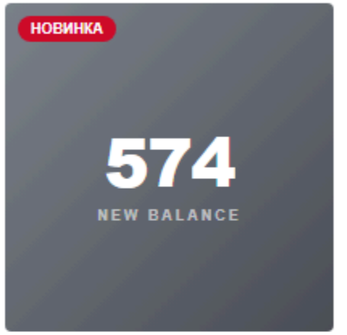
ID покупця

1

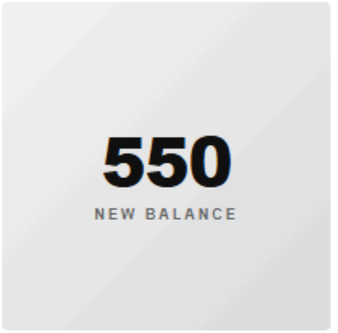
Замовлення #12 оформлено!

Помилка запиту: 401

Картка товару



New Balance 574
Розмір 42 · Сірий
3 299 ₴



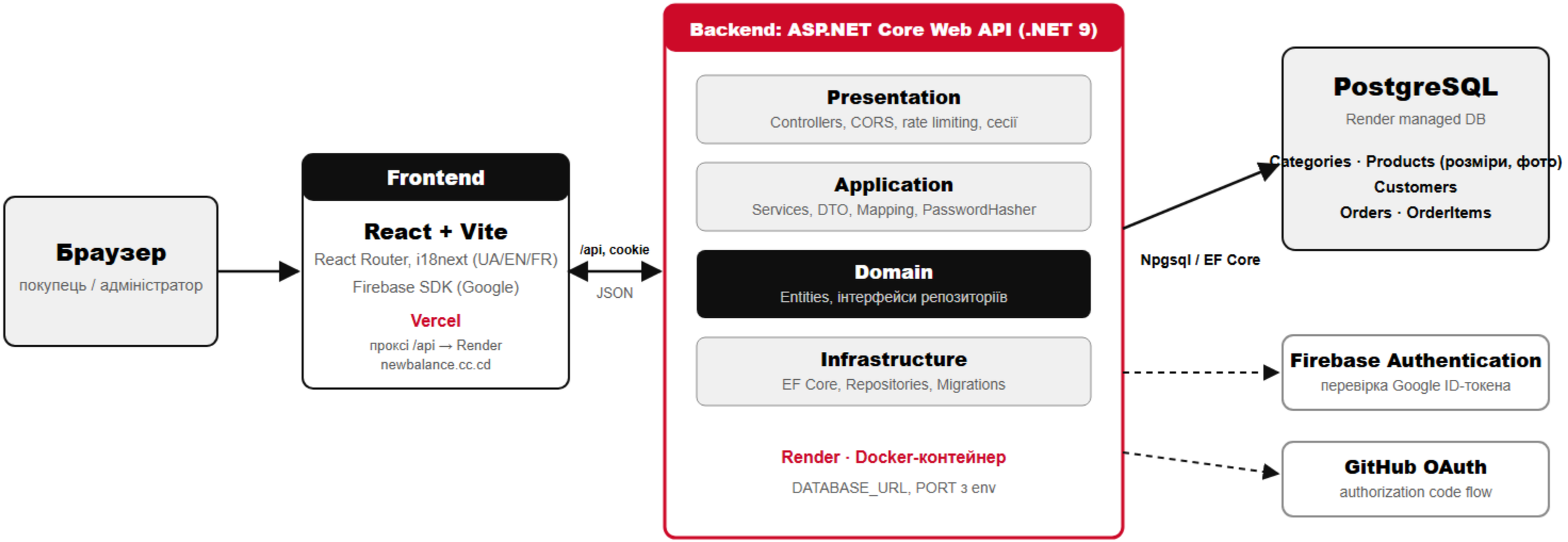
New Balance 550
Розмір 43 · Білий
4 199 ₴

Фото товарів замінено типографськими плитками: немає прав на офіційні фото бренду.

Архітектура системи і розміщення

Фронтенд, бекенд, база даних та зовнішні сервіси

New Balance Shop

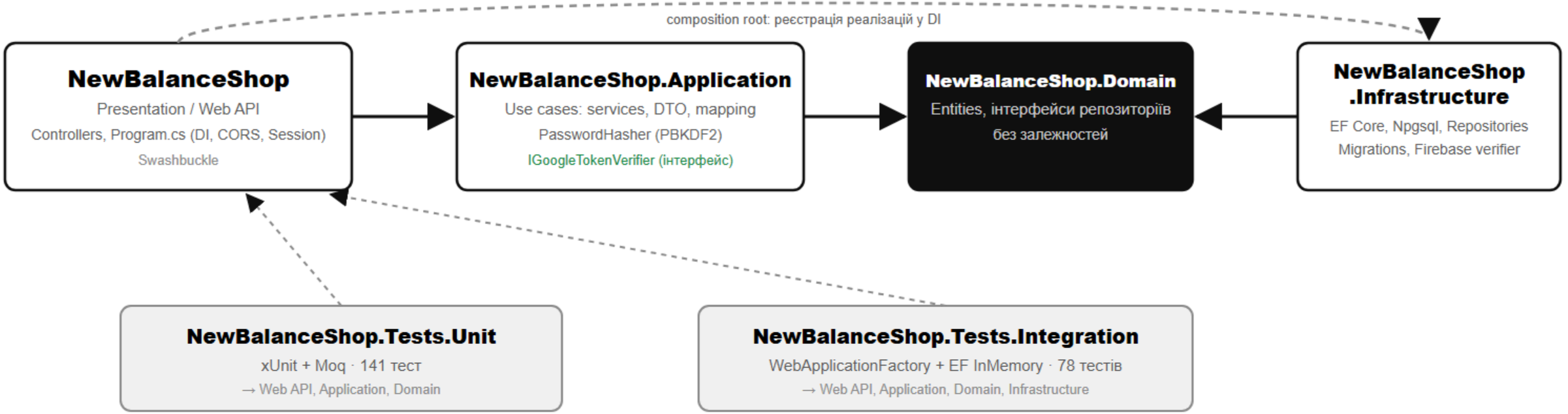


Адреси: сайт newbalance.cc.cd (запасні: newbalanceshop.vercel.app, nb.rafail.xyz), API newbalanceshop-api.onrender.com. Запити /api проходять через Vercel, тож сесійна кука належить сайту.

Чиста архітектура: проєкти та зв'язки

4 проєкти рішення NewBalanceShop.slnx + 2 тестові · стрілка = ProjectReference (залежність)

New Balance Shop



Правило залежностей: усі стрілки між шарами спрямовані до Domain

Що дотримано

- Domain не посилається ні на що (немає ProjectReference).
- Infrastructure реалізує контракти з Domain і Application, самі шари від неї не залежать.
- Controllers викликають лише сервіси Application, EF Core у них немає.

Відхилення виправлено

Раніше Application мав пакет FirebaseAdmin. Тепер перевірка Google-токена йде через інтерфейс **IGoogleTokenVerifier**, а реалізація **FirebaseGoogleTokenVerifier** лежить в Infrastructure.

Використані патерни

Кожен патерн розв'язує конкретну задачу проєкту

Repository

`IRepository<T> → Repository<T>`

Доступ до даних сховано за інтерфейсом у Domain.

```
public interface IProductRepository
  : IRepository<Product>
{ Task<List<Product>> GetByCategoryAsync(int id); }
```

Навіщо: сервіси не знають про EF Core; у юніт-тестах репозиторій підміняється Moq.

Dependency Injection

`Program.cs`

Усі залежності передаються через конструктор, реєстрація в одному місці.

```
builder.Services
 .AddScoped<IProductService, ProductService>();
```

Навіщо: слабка зв'язаність; в інтеграційних тестах підміняємо БД на InMemory.

Service Layer (Use Cases)

`Application/Services`

Бізнес-логіка в сервісах, контролер лише приймає й повертає HTTP.

```
OrderService.CreateAsync:
 перевірка складу, UnitPrice,
 TotalPrice, зменшення Stock
```

Навіщо: логіку можна тестувати без HTTP і без БД.

DTO + Mapper

`Application/DTO, Mapping`

API повертає DTO, а не сутності EF; мапінг у extension-методах.

```
public static CustomerDto ToDto(
  this Customer c) => new() { … };
```

Навіщо: PasswordHash ніколи не потрапляє у відповідь (є тест на це).

Middleware Pipeline

`Program.cs`

Запит проходить послідовність компонентів.

```
UseSwagger → UseCors
 → UseSession → MapControllers
```

Навіщо: CORS із cookie й сесія для кошика та логіну налаштовані централізовано.

Session State

`CartController, AuthController`

Кошик гостя й ID залогіненого покупця живуть у серверній сесії.

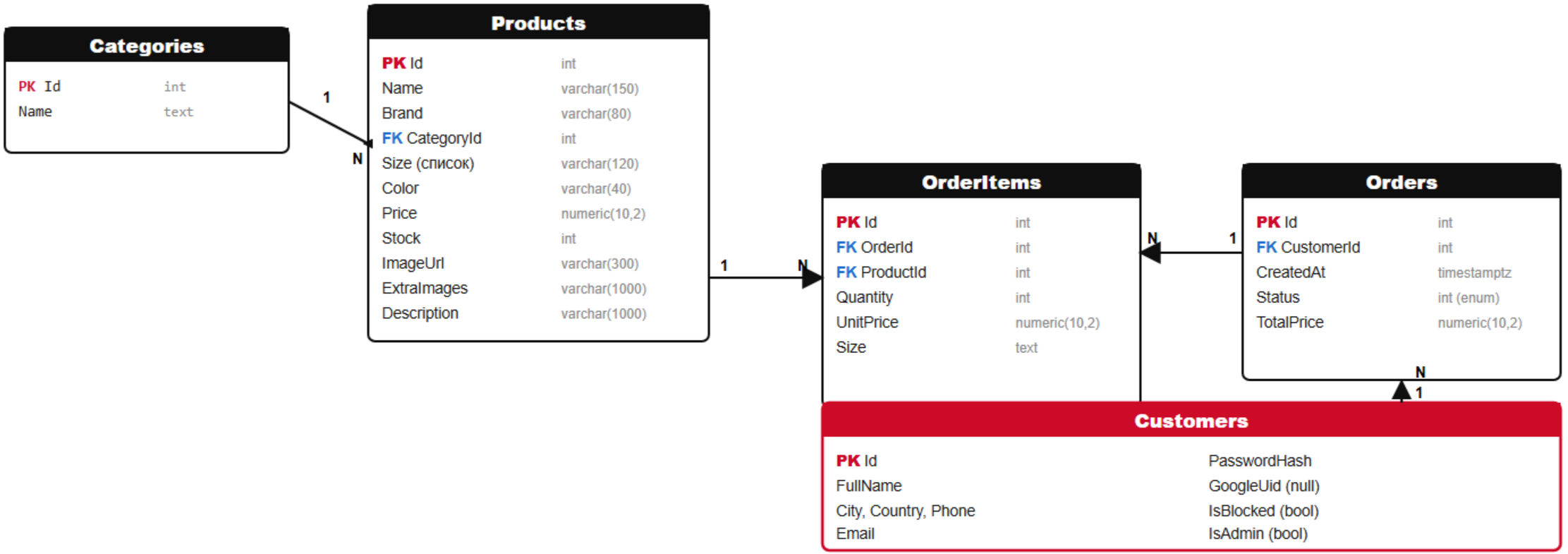
```
HttpContext.Session
 .SetInt32("customerId", id);
```

Навіщо: кошик не пишемо в БД до оформлення; логін без токенів.

База даних: ER-діаграма

PostgreSQL (Render) · 5 таблиць · EF Core Migrations + seed

New Balance Shop



Чому PostgreSQL

- Реляційні зв'язки замовлення → позиції → товар, потрібна транзакційність.
- Безкоштовний managed-інстанс на Render, є провайдер Npgsql для EF Core.

Структура

3НФ: ціна позиції (UnitPrice) фіксується на момент замовлення; Customer не дублює дані замовлень. Індекси на всіх зовнішніх ключах, унікальний Email, видалення покупця із замовленнями заборонено (Restrict).

Що покращити

Залишок зараз спільний для всіх розмірів: наступний крок — окрема таблиця розмірів із залишком на кожен. Ще м'яке видалення товарів.

Authentication / Authorization та безпека

Як зберігаємо паролі, як перевіряємо права і які ризики закрито

New Balance Shop

Аутентифікація: 3 способи входу

1. Email + пароль: `/api/Auth/register, login`
2. Google: Firebase ID-токен, API перевіряє підпис
3. GitHub OAuth (authorization code flow)

→ Сесія: cookie HttpOnly, Secure, таймаут 30 хв

```
Пароль: PBKDF2-HMAC-SHA256
100 000 ітерацій · сіль 16 байт · хеш 32 байти
формат: ітерації.сіль.хеш · FixedTimeEquals
Google-користувачі: пароль не зберігається
```

Авторизація і захист

ЗАГРОЗА	ЗАХИСТ	
SQL Injection	EF Core, параметризовані запити	**OK**
XSS	React екранує вивід	**OK**
Паролі	Hash + salt, не SHA256(password)	**OK**
Адмін-функції	Перевірка IsAdmin: 401 / 403	**OK**
Чужий профіль	Тільки власник або адмін	**OK**
Секрети	DATABASE_URL, Firebase JSON у env	**OK**
BOLA / IDOR	customerId лише із сесії; чужі замовлення: 403/404	**Закрито**
CRUD товарів	[RequireAdmin]: 401 / 403	**Закрито**
CORS / сесія	Білий список origin, SameSite, /api через домен сайту	**Закрито**
Підбір паролів	Rate limiting: 10 спроб на хвилину	**Закрито**
Пароль адміна	У репозиторії лише хеш, пароль змінено	**Закрито**

Що закрито за цей час

- **BOLA:** покупець береться із сесії в /api/Orders і checkout.
- **Товари:** запис лише для адміністратора.
- **CORS:** білий список сайтів замість «усім».
- **Email:** унікальний індекс у БД; покупця із замовленнями видалити не можна.
- **Brute-force:** rate limiting на login і реєстрацію.
- **Секрет:** пароль адміна прибрано з коду.
- **Архітектура:** Firebase за інтерфейсом.

Ще в плані: сесії в Redis/БД, звіт ZAP.

Testing & Logging

xUnit · Moq · WebApplicationFactory · EF InMemory · coverlet

New Balance Shop

Піраміда тестів

219 / 219 passed

Integration · 78
повний HTTP-конвеєр, EF InMemory

Unit · 141
сервіси, мапери, контролери (Moq)

Unit (NewBalanceShop.Tests.Unit)
Product/Order/Auth/CustomerService · PasswordHasher · ProductSizes · Mapping · контролери (Products з фільтрами, Orders, Auth, Customers, GithubAuth)

Integration (NewBalanceShop.Tests.Integration)
Products, Cart, Orders, Auth, Customers: розміри, залишок, права 401/403, захист від BOLA; кожен тест на власній InMemory-БД

Покриття: 91,1%
Infrastructure 89,5% · Application 99,7% · Domain 87,8% · Web API 85,6%

Що знайшли тести (реальні баги)

- **Forbid() давав 500** замість 403, бо схему автентифікації не реєстровано; тепер явні 401/403.
- **Secure-cookie ламала сесію** у тестах по http; у середовищі Testing вимкнено.
- **Два провайдери БД** при підміні Npgsql на InMemory; усунуто видаленням конфігурації контексту.

Logging

Вбудований **ILogger** і глобальний **ExceptionHandlingMiddleware**: непередбачені помилки логуються, клієнт отримує коректну відповідь. Логи доступні в Render.

План покращення

- Запуск тестів у GitHub Actions на кожен push (workflow готовий, потрібне право workflow для токена).
- Звіт сканера ZAP для додатка.

Ключові метрики

Фактичні цифри на 14.10.2026, виміряно з репозиторіїв і прогону тестів

New Balance Shop

13/13
ВИМОГ SRS ПОВНІСТЮ

33
REST-ЕНДПОІНТИ

219
ТЕСТІВ (141 UNIT + 78 INTEGRATION)

91,1%
CODE COVERAGE (РЯДКИ)

78
КОМІТІВ (БЕКЕНД 56 + ФРОНТЕНД 22), 18 MERGE-КОМІТІВ

4 тиж.
РОЗРОБКИ: 14.09 – 14.10

2,0k
РЯДКІВ C# У БЕКЕНДІ (БЕЗ ТЕСТІВ І МІГРАЦІЙ)

3,0k
РЯДКІВ JS/CSS У REACT-ФРОНТЕНДІ

Покриття тестами за проєктами (unit + integration, злиті звіти coverlet)



1090 рядків, 993 покрито. Без міграцій і тестових проєктів. Команда: dotnet test --collect:"XPlat Code Coverage".

Scrum: спринти, беклог, канбан

Jira Scrum board SCRUM · newbalance-team1.atlassian.net

New Balance Shop

Спринти

СПРИНТ	ДАТИ	ЦІЛЬ
1	до 14.09	SRS, архітектура, CRUD
2	31.08–06.09	Початок розробки, SCRUM-ролі
3	07.09–13.09	Замовлення, Customer
4	14.09–20.09	Деплой, фронтенд, i18n
5	21.09–27.09	MVP, авторизація, 4 проєкти
6	28.09–04.10	Кабінет, адмін, тести

Team Lead — Панчук Вадим; команда розробки — обидва учасники.

Канбан-дошка (стан на 14.10.2026)

BACKLOG 4

- CI: GitHub Actions (workflow готовий)
- Кешування категорій (IMemoryCache)
- Звіт ZAP (безпека)
- Залишок по кожному розміру

IN PROGRESS 2

- Презентація і сценарій доповіді
- Jira: спринти і внесок команди

DONE 16

- SRS, UML, архітектура
- CRUD товарів, замовлення, кошик
- PostgreSQL + Docker + Render
- React-фронтенд (Vercel), UA/EN/FR
- Clean Architecture: 4 проєкти
- Авторизація: пароль, Google, GitHub
- Кабінет, адмін-функції (FR-10–13)
- Фільтри, пошук, сортування (FR-03)
- React Router: /order, /fav, /product
- Форми входу і реєстрації
- Адмін-панель у React
- Розміри і галерея товарів
- Безпека: BOLA, CORS, rate limit
- 219 автотестів, покриття 91,1%
- Помилки: глобальний middleware
- Scalar, DocFX, README

Tools & Workflow

Інструменти розробки, деплою та командної роботи

New Balance Shop

БЕКЕНД І ДАНІ

- .NET 9 / ASP.NET Core
- PostgreSQL
- EF Core + Npgsql
- Swagger / Scalar
- Firebase Auth

ФРОНТЕНД

- React
- Vite
- i18next (UA/EN/FR)

ТЕСТУВАННЯ

- xUnit
- Moq
- WebAppFactory
- coverlet

ПРОЦЕС І ДЕПЛОЙ

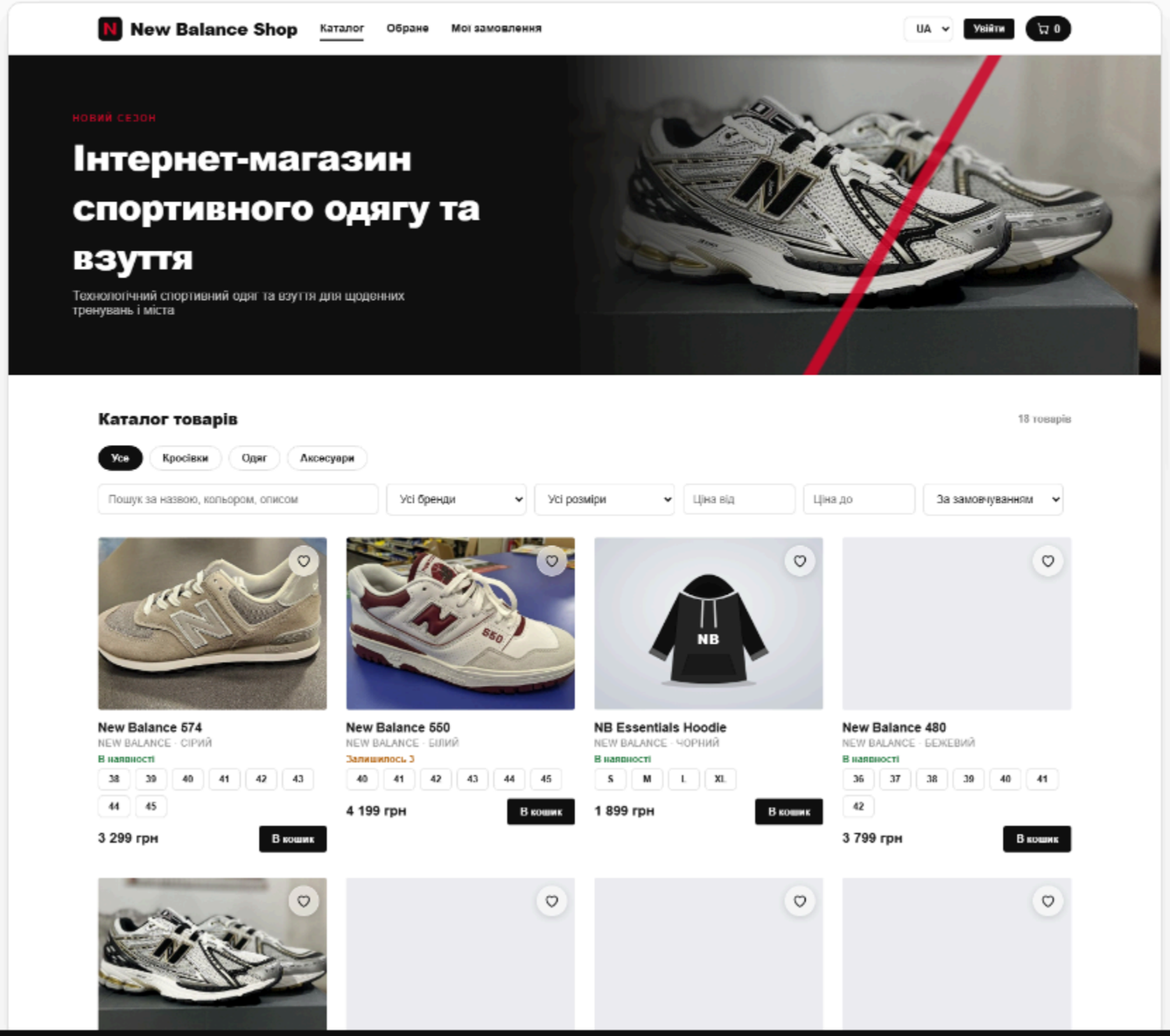
- GitHub
- Jira (Scrum)
- Docker
- Render (API + БД)
- Vercel (фронтенд + проксі /api)

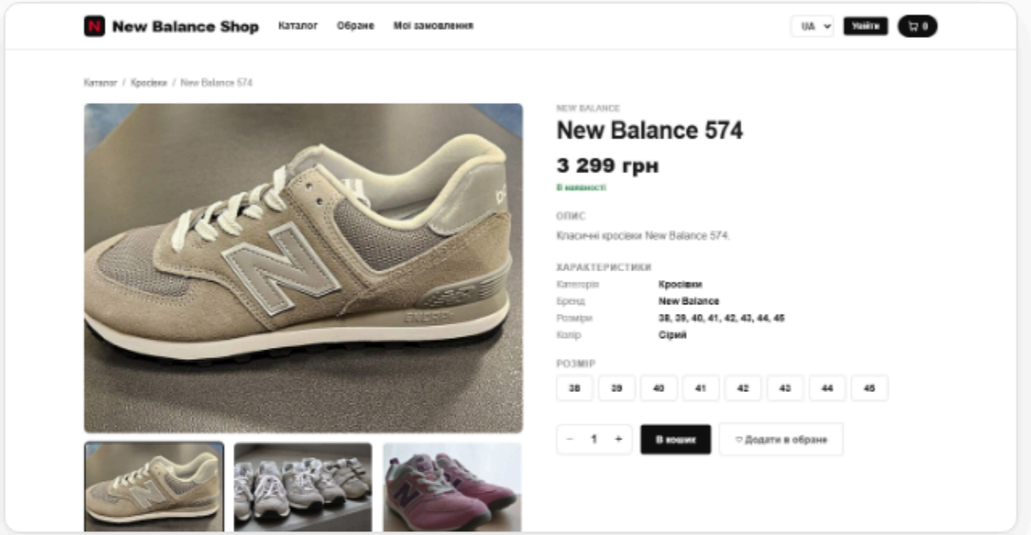
Код → **git push** → **GitHub** → **Render збирає Docker** / **Vercel збирає React** → **онлайн**

Каталог і картка товару

Живий сайт newbalance.cc.cd · 18 товарів, реальні фото, розміри, фільтри

New Balance Shop





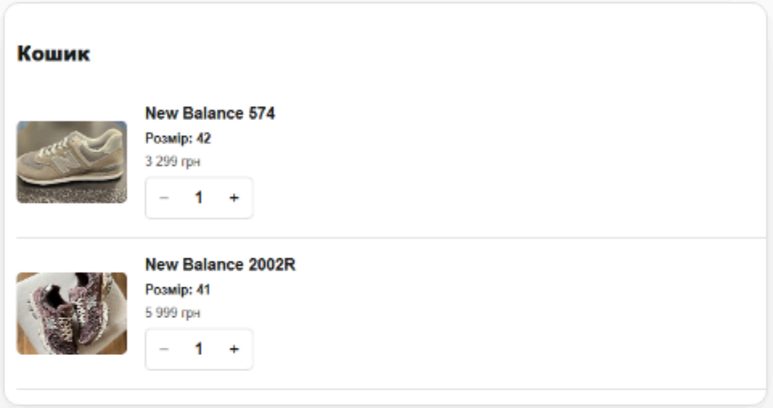
Що є на сайті

- **Каталог:** категорії, пошук, бренд, розмір, діапазон цін, сортування. Фільтри зберігаються в адресі, ними можна ділитися.
- **Картка товару** `/product/:id`: галерея з трьох кадрів і вибір розміру.
- **Розмір** обирається прямо на картці каталогу, кнопка «В кошик» додає одразу.
- **Фото:** вільні ліцензії (CC0 / CC BY) з Wikimedia, автори в `CREDITS.md`.

Кошик, оформлення, обране

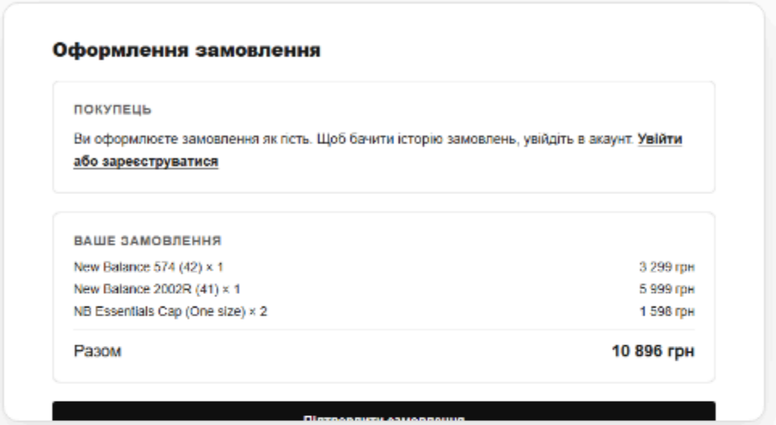
Сценарій покупця без обов'язкової реєстрації · адреси /cart, /order, /fav

New Balance Shop



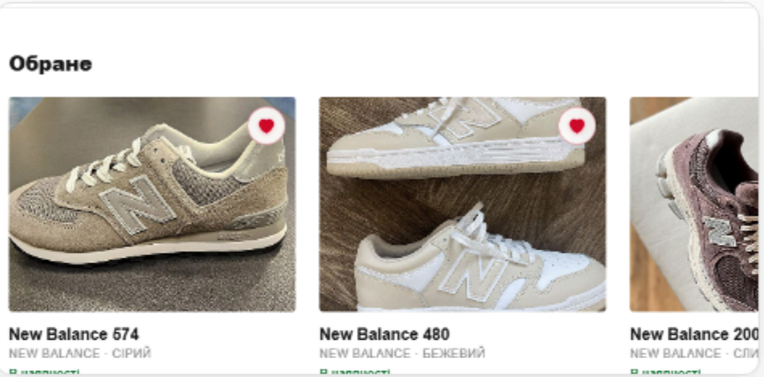
Кошик /cart

Позиції «товар + розмір», зміна кількості, залишок на складі перевіряє сервер, очищення через модальне вікно.



Оформлення /order

Покупець береться із сесії, гість оформляє на службового «Гостя». Підсумок і підтвердження в один крок.



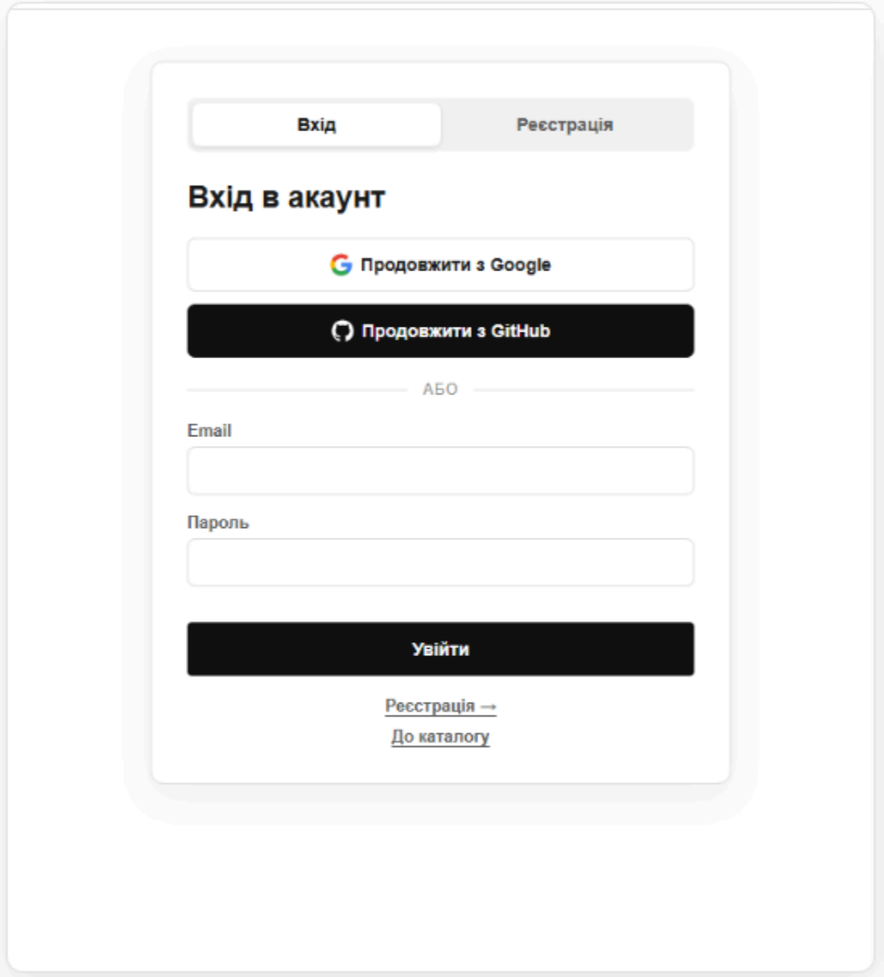
Обране /fav

Сердечко на картці, список зберігається в браузері, лічильник у шапці.

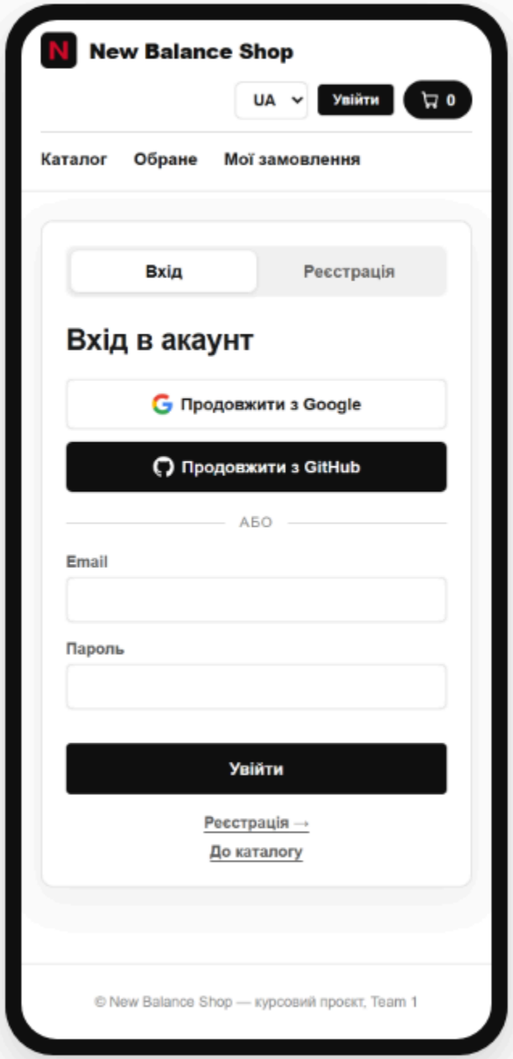
customerId береться із сесії (захист від BOLA) · залишок спільний для всіх розмірів · історія замовлень: /orders · кабінет: /account

Вхід і реєстрація

Email + пароль, Google, GitHub · одна сторінка з перемикачем «Вхід / Реєстрація»



Комп'ютер



Телефон

Три способи входу

- **Email і пароль:** PBKDF2, 100 000 ітерацій, сіль.
- **Google:** Firebase ID-токен, підпис перевіряє сервер.
- **GitHub:** OAuth, після входу створюється акаунт покупця.

Сесія і куки: проксі /api

Сторінка та API на різних доменах, браузери блокують «сторонні» куки. Рішення: запити йдуть на той самий домен, а Vercel проксує `/api` на сервер.

Браузер → **newbalance.cc.cd/api**

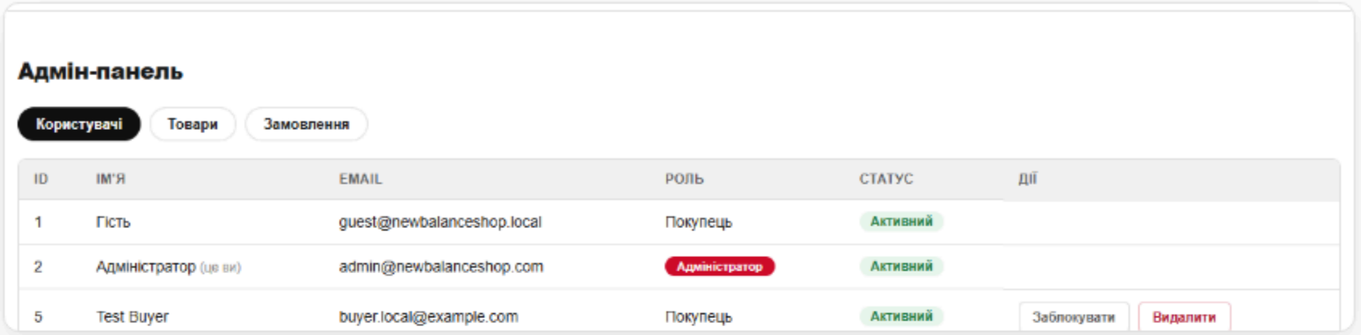
Vercel rewrite → **Render API**

Кука належить сайту й не блокується

Адмін-панель модератора

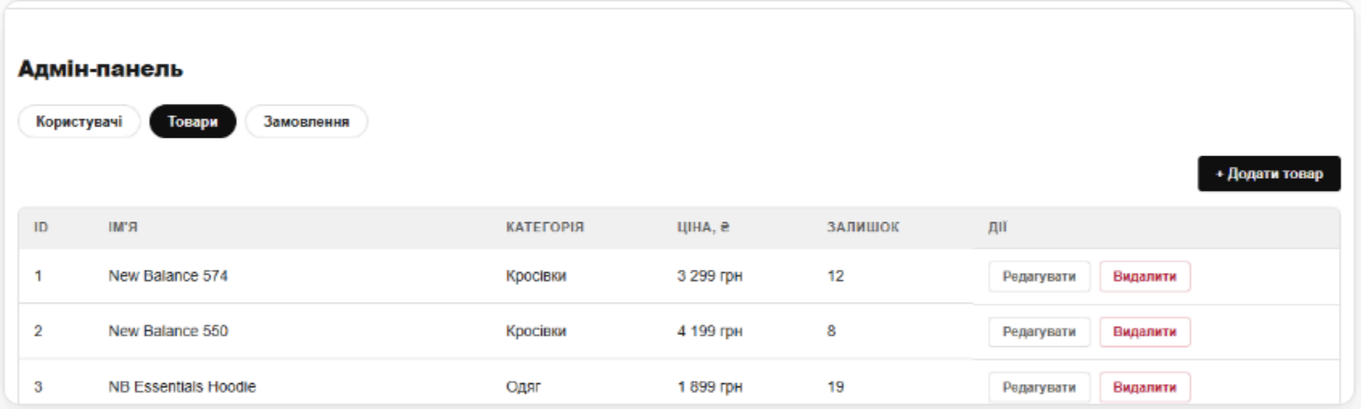
Сторінка /admin доступна лише ролі адміністратора · сервер повертає 401 / 403

New Balance Shop



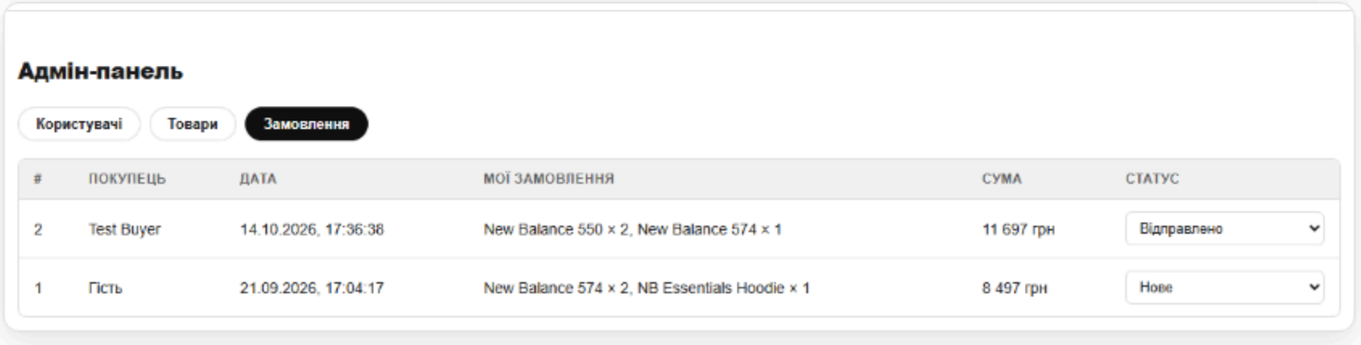
Користувачі (FR-12, 13)

Список, блокування і розблокування, видалення з підтвердженням. Користувача із замовленнями видалити не можна, лише заблокувати.



Товари (FR-04–06)

Додавання, редагування й видалення: ціна, залишок, розміри через кому, головне й додаткові фото.



Замовлення (FR-09)

Усі замовлення магазину від нових до старих, зміна статусу: New, Confirmed, Shipped, Delivered, Cancelled.

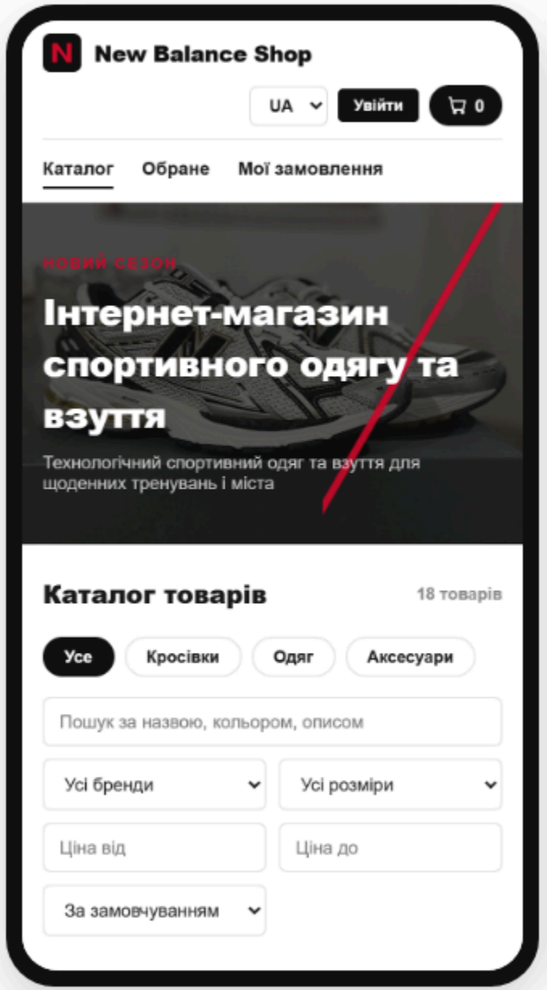
пароль адміна не лежить у репозиторії, лише PBKDF2-хеш

запис товарів: тільки IsAdmin

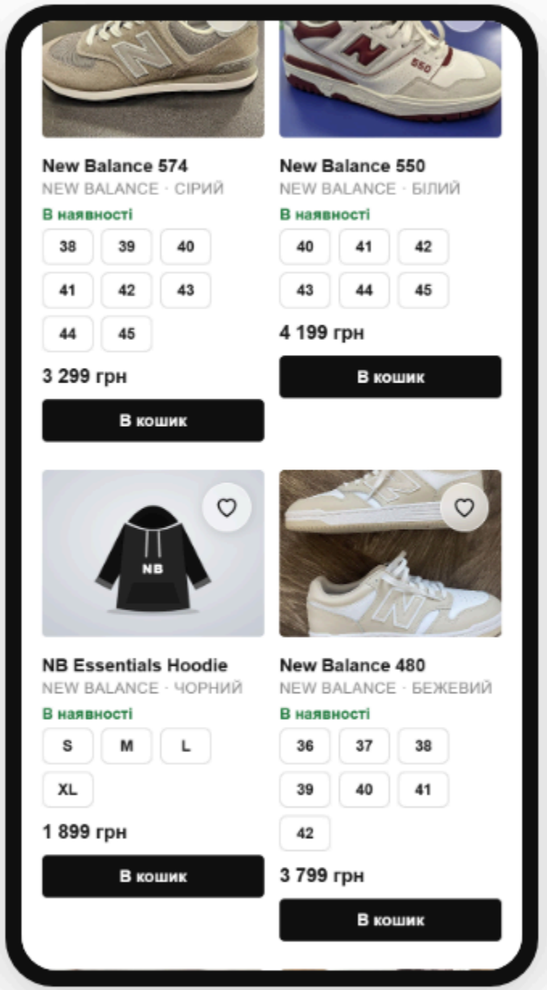
службовий «Гість» без кнопок блокування

Мобільна версія

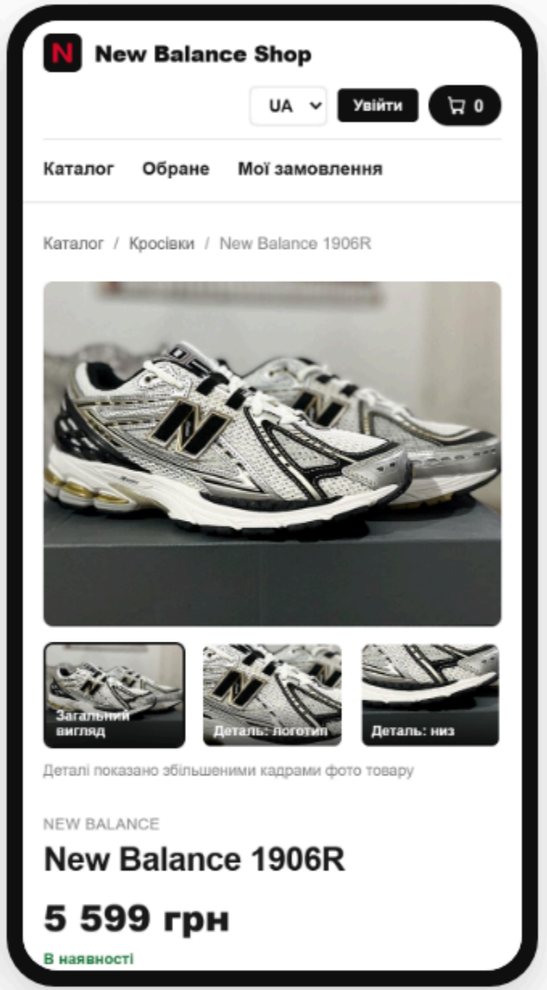
Знімки адаптивної версії 390 px · шапка переноситься, товари в дві колонки, розміри на картці



Каталог і шапка



Картки з розмірами



Картка товару

Що зроблено для телефона

- Шапка переноситься на два рядки, меню прокручується пальцем.
- Сітка товарів у дві колонки, розміри обираються на картці.
- Адмін-таблиці на вузькому екрані стають картками.
- Жодного горизонтального скролу.
- Знімки зроблено на живому коді, не в макеті.

Портфоліо-цінність

Чому проєкт важливий для розробників, потенціал для аудиторії та монетизація

1

Для нас як розробників

- Повний цикл: вимоги, архітектура, БД, API, фронтенд, деплой.
- Clean Architecture із 4 проєктів і 219 автотестами.
- Авторизація трьома способами: пароль, Google, GitHub.
- Працює онлайн, є що показати роботодавцю.

2

Потенціал для ЦА

- Швидка покупка без обов'язкової реєстрації.
- Кабінет: профіль та історія замовлень.
- Інтерфейс трьома мовами: UA, EN, FR.
- Каталог легко розширити: нові категорії, бренди, фільтри.

3

Можливості монетизації

- Партнерська комісія з продажів бренду.
- Платна доставка й прискорене оформлення.
- Інтеграція оплати (картка, Apple/Google Pay).
- Рекламні й промо-місця для нових колекцій.
- Шаблон для малого бізнесу: готовий магазин «під ключ».

КЛЮЧОВА ДУМКА ПРОЄКТУ

Від каталогу до замовлення за кілька кліків, без зайвої реєстрації.

Ваші питання?

New Balance Shop · Team 1 · дякуємо за увагу

Живий сайт
newbalance.cc.cd

Код (GitHub)
github.com/vicitmceo/NewBalanceShop

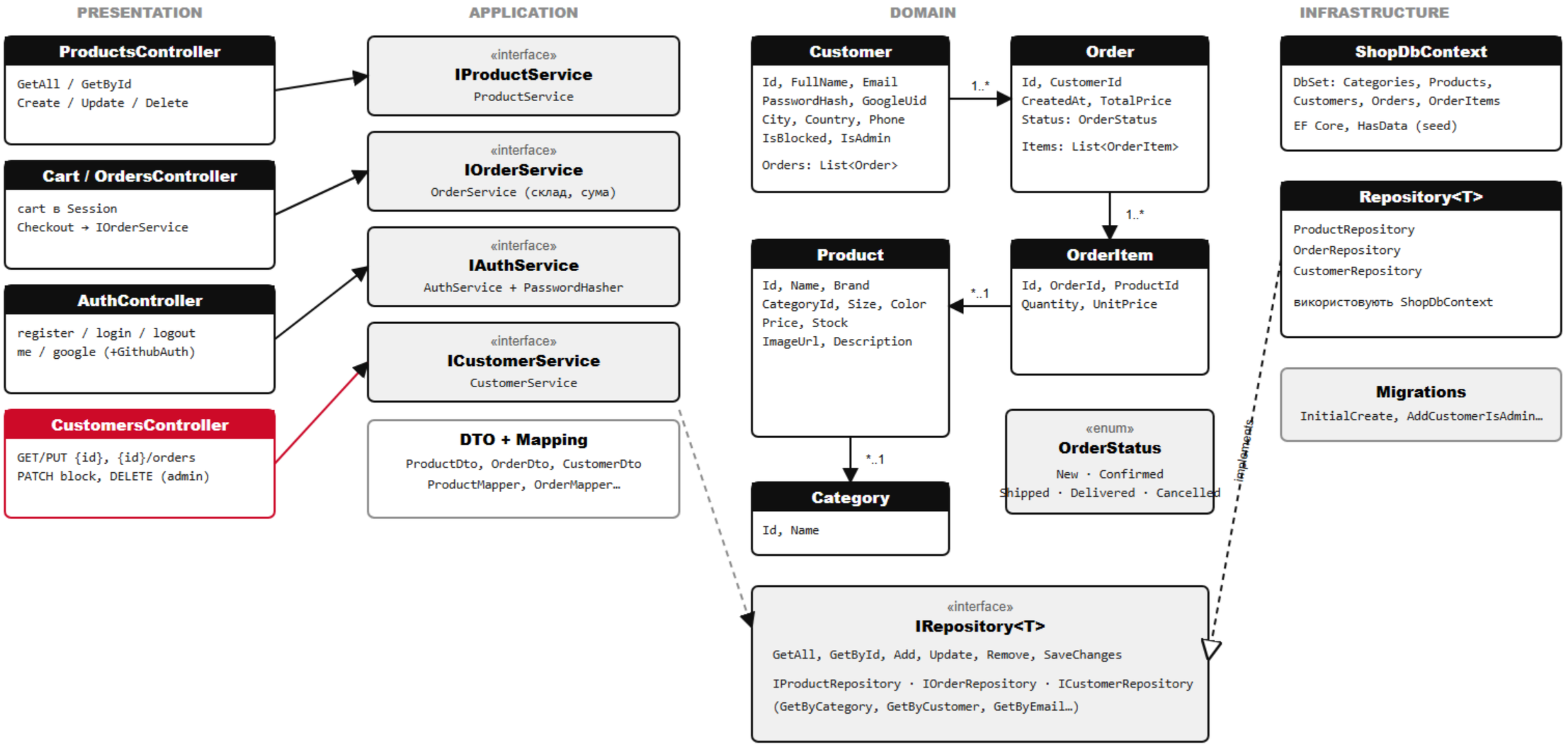
Документація
vicitmceo.github.io/NewBalanceShop

Панчук Вадим
alfa40957@gmail.com
Ноцколіні Рафаїл
vicitmceo@gmail.com

Діаграма класів

Ключові класи й інтерфейси за шарами (спрощено)

New Balance Shop

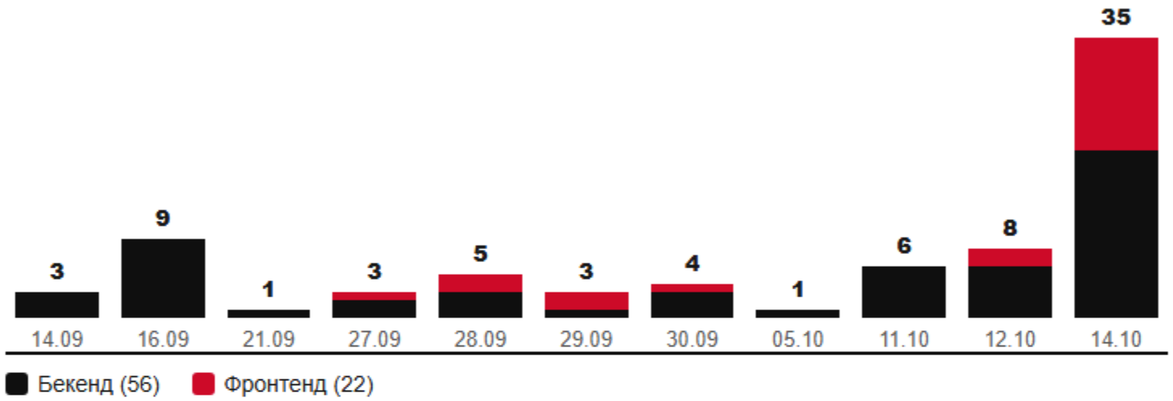


Командна робота в Git

Коміти, гілки та внесок учасників · репозиторії бекенду й фронтенду

New Balance Shop

Коміти за днями (усього 78)



Гілки і merge (бекенд, фронтенд)

ГІЛКА	ЩО ЗРОБЛЕНО
`feature/security-hardening`	BOLA, CORS, rate limiting
`feature/catalog-and-admin-api`	фільтри, адмін-API замовлень
`feature/sizes-and-gallery`	розміри, галерея, 18 товарів
`fix/first-party-session-…`	вхід через GitHub без сторонніх кук
`feature/spa-routing-and-admin`	маршрути, адмін-панель (фронтенд)

Нова робота йде у feature- і fix-гілках з merge-комітом у master: 12 merge-комітів у бекенді, 6 у фронтенді. Старий код 14.09–05.10 писався прямо в master.

Внесок учасників

УЧАСНИК	РОЛЬ	КОМІТИ	ЩО ЗРОБЛЕНО
Ноцколіні Рафаїл	Full-stack розробник	**78 (100%)**	Бекенд: архітектура, API, БД, авторизація, безпека, тести; фронтенд React (маршрути, адмін-панель); деплой; документація
Панчук Вадим	Team Lead	**0**	Запрошений у Jira як учасник; коміти в репозиторії відсутні

⚠ Внесок Team Lead у планування, Jira-дошку й рев'ю не відображено в git. Доповнити вручну перед захистом.

Документація

README, документація API (Scalar / OpenAPI), архітектурні й користувацькі документи

New Balance Shop

Що є в репозиторії

ДОКУМЕНТ	ПРИЗНАЧЕННЯ	СТАН
README.md	Опис за шаблоном викладача, стек, запуск, скріни	є
SRS.md	Вимоги, use cases, NFR, матриця трасування	є
docs/ARCHITECTURE.md	Clean Architecture, UML класів	є
docs/STRUCTURE.md	Структура бекенду, фронтенду, БД	є
docs/api/index.html	Документація REST API на Scalar з OpenAPI	є
Swagger UI (/swagger)	Інтерактивний API-провайдер на Render	є
docs/user_guide.html	Інструкція для покупця	є
docs/test_report.html	Звіт про тестування	є
DocFX на GitHub Pages	Документація публічних методів із XML-коментарів (docs/code)	є

```
NewBalanceShop/
├─ README.md · SRS.md · TEAM.md · BACKLOG.md
├─ docs/
│  ├─ ARCHITECTURE.md · STRUCTURE.md
│  ├─ api/  index.html (Scalar) + openapi.json
│  ├─ architecture/  структура back/front/DB
│  ├─ db/  postgres-tables.png
│  ├─ user_guide.html
│  └─ test_report.html
├─ defense-materials/  слайди до захисту
└─ Dockerfile · render.yaml
```

Документація API

Специфікація OpenAPI генерується Swashbuckle, HTML-сторінка побудована на Scalar. Ендпоінти згруповані: Products, Cart, Orders, Auth, Customers.